CSCE3304 Digital Design II – Fall 2026

Finding Cells Worth Adding With AI Used as a Development Aid

GB3:
Kareem Rashed (900233328)
Mahinour Abdelgawad (900231870)
Joshua Makar (900231109)

GitHub Repository URL: https://github.com/Kareem-Rashed/celladvisor
Final Commit Hash :

1. How we used AI

We used AI as a development aid: an assistant helped us understand the handouts and write and test our code, but no AI model runs when our tool runs, so the same input always gives the same output. Before writing any code we wrote AGENTS.md, which tells the assistant how the project is checked and what it must never change. Every use whose output we kept is logged in ai/use_log.md, every AI mistake our checks caught in ai/mistakes_log.md, and every AI-assisted commit ends with an *AI-assisted: tool/model* trailer.

2. What the AI did and what we did, part by part

Part	Member	What the AI did	What the member did	Log
AGENTS.md	Kareem	Drafted it from the overview and handout	Read it against overview sections 4.1 and 4.3 and kept it as is	L001
Canonical form	Kareem	Explained the concepts, wrote the tests	Wrote canon.py	L002
AIG reader, cut enumerator, scripts	Kareem	Explained the AIGER format and cut enumeration, wrote the test files, the AIG and Table 1 scripts, the Make targets, the cut-count script and the area model (step by step)	Wrote aig_reader.py and cutter.py himself from the explanations; reviewed every script; chose which cells to leave out of the area model	L010–L016
Liberty parser, cut checker, redundancy checker, cost	Mahinour	Wrote first versions of each	Adjusted each to fit the repo (canon.py, verify_cut, a test main block and path fix, the cost docstring); ran the tests; found the cost/area-model mismatch by reading both modules together	L003, L007–L009, L017

Part	Member	What the AI did	What the member did	Log
Contained test, broken-case tests, interface doc	Joshua	Explained the concepts and wrote the code and tests	Kept the algorithm; moved it to src/ and made it read cells through hdlib.py, adding the area	L004–L006
Liberty writer, requests, Table 5 with cec	Joshua	Wrote the code, tests and broken cases	Reviewed and tested it; asked for a second Yosys read after finding that read_liberty -lib accepts an unknown pin; fixed the two mapping bugs M005 and M006	L018–L019
Greedy selection, generator areas, OpenSTA, demo	Kareem	Wrote the later versions of the greedy selection (script B totals, parallel trials, a rule that every chosen cell must appear in a netlist), the OpenSTA delay script and the make demo command. Helped explain the generator areas script.	Typed real_areas.py and total_area himself, reviewed the AI's greedy versions, and investigated why unused cells change the mapped area. Checked the baselines against the checkpoint 2 tables and the first five generated cells against manual runs.	L020
30% Test, Table 5 for final cells	Joshua	Explained the 30% experiment and wrote sensitivity.py, its test and make sensitivity. Made table5.py accept any library and output	Agreed the setup before any code (shrink the first greedy cell, measure with script B like the	L022, L023

Part	Member	What the AI did	What the member did	Log
		folder, wrote final_real_areas.py, and added make table5-final and make table5-real.	greedy). Reviewed the diff before committing. Checked that the unchanged final set reproduces the greedy total exactly, that both libraries load with 199 cell types and that all mappings pass cec.	
Cell-by-cell curve	Mahinour	Wrote the first version of curve.py	Changed it to include spm and mul	L021

3. Tools, models and cost

Tool and model	Used by	Used for	Cost
Claude, Opus 5.5	Kareem, Joshua	AGENTS.md, explaining concepts, tests and broken cases, the Liberty writer, Table 5 and the 30% test	Subscription: Pro
Claude, Sonnet 5.5	Mahinour, Kareem	Liberty parser, cut checker, redundancy checker, cost, cell-by-cell curve, the greedy selection and real-area scripts, OpenSTA and the demo.	Subscription: Pro (Kareem) Free Tier (Mahinour)
GitHub Copilot, MAI-Code-1.1-Flash	Mahinour	Updating documentation	Free Tier

4. How we described tasks and checked the output

4.1. Describing Tasks

We gave the AI the exact requirement from the handout (for example Table 3's definition of Contained, or section 6's Liberty format), our shared conventions from docs/interfaces.md (truth-table bit order, module layout), and the files it had to fit. We would also ask the AI to explain a concept before writing any code (for example the AIGER format, L010, and cut enumeration, L011), and then write the module ourselves from the explanation, or ask for one piece at a time (the area model, L016). The assistant's rules are in AGENTS.md: pass or fail comes from the check scripts, every number in a report comes from a file in results/, and the assistant never changes the check scripts, the broken cases or the PDK.

4.2. Checking output

A result is counted only if it passed a check based on the handout, never on the AI's word:

- the redundancy checker must reproduce the handout's Table 4 exactly (24 / 20 / 38 / 3,822);
- the Liberty writer must reproduce the handout's example cell exactly, and its request must be 0x077F, as in Appendix A;
- Table 1 areas compared with the handout's; baseline areas compared with given/scripts/map.sh;
- every mapping must pass ABC's cec;

5. How we applied the rules of section 4.3

Rule	How we applied it
1. Pass or fail comes from check scripts	Every check is a test in make test; nothing was accepted on the AI's say-so.
2. Every number comes from a file the flow wrote	All report numbers come from files in results/, written by make targets.
3. Every check fails on a broken case	Broken cases in tests/broken/: a one-bit-wrong cut truth table, an HD cell that must be Exists, two invalid Liberty files, a netlist with a swapped gate, [others].
4. AI must not change checks, broken cases or the PDK	AGENTS.md forbids the AI from editing tests/broken/, the check scripts, given/ and pdk/. We reviewed each diff before committing (as logged, for example L023). The PDK and given files are linked by make setup, never copied
5. Every member can explain every part of the code	Walked through each other's code upon merging and asked each other to explain parts that are unclear.

6. The five most instructive AI mistakes

ID	What was wrong	How it was caught	Would a weaker check have missed it?
M005	Writing the reference netlist before abc changed ABC's result (cpu8, script A: 4,209.04 instead of 4,181.51 μm²)	Comparing one run with given/scripts/map.sh	Yes: every mapping still passed cec and the areas looked plausible
M006	cec matched flip-flops by name, which differ between the two netlists, so a correct mapping failed	cec failed on cpu8, which maps correctly	No: it failed clearly. The fix was re-checked against a swapped gate
M002	cost.py's single-stage test differed from areamodel.py's	Reviewing the two modules against each other	Yes: both passed their own tests
M001	The Contained code assumed a folder layout and a parser the repository did not have	Running it in the real repository	Yes: it passed its own unit tests
M007	The first curve.py only looked in given/designs/, so spm and mul were not found	The script threw an exception when run	No: it failed loudly.

7. A decision we would never hand to an AI

We would never let an AI decide whether a measured area reduction is real. The AI optimizes what it is shown, and it reported savings that were not savings. Checkpoint 2's best-of-both reduction of 1.70% looked like a result, but split by script it was -1.79% for script A (which barely uses the cells) and +0.14% for script B, which does; the saving came from script A's area changing with the library, not from the cells. Later, the first greedy run chose five cells of which four never appeared in any netlist, and the AI's own control experiment moved by 1,400 μm² (M008, M009).

Our rule, which we set ourselves: a reduction counts only if (1) the new cells appear in the mapped netlists, (2) it exceeds the noise we measured (about 250 μm² for script B), and (3) it holds for the best of both scripts, the area a designer gets.

8. Where AI saved time and where it cost time

Saved: Learning the AIGER and Liberty formats and the Yosys/ABC commands, and writing tests, broken cases, scripts and Make targets.
Cost: Wrong assumptions about our repo (M001), debugging the cec and baseline setup (M005, M006), reconciling two modules (M002), and reviewing code we didn't write.

9. Next time

Would hand to an AI: explanations, test drafts, boilerplate scripts and Make targets.

Would not: (1) deciding whether a measured saving is real, or how to measure it (which script counts, what the control is, how big the noise is; (2) any number that does not come from a file in results/.

Why: the AI optimizes the number it is shown. It reported a 1.70% saving that was script A's noise, picked four cells that never appeared in a netlist, and called script B stable. And the rule that every member must explain every part of the code cannot be met by code we only reviewed.